GPU Related Concepts

PCB - Printed circuit Board (PCB) is a non-conductive, rigid board (usually fiberglass or epoxy) that serves as the mechanical foundation for electronic components. It connects components (CPU, memory, resistors) using copper tracks etched into the surface, allowing them to communicate.

GA102 — In computer architecture stands for Graphics Ampere 102. It is a specific GPU silicon die from NVIDIA, based on Ampere microarchitecture, and is primarily used in high-end RTX 30-series graphics cards like RTX 3080 and RTX 3090

G = Graphics

A = Ampere Architecture

102 = The designation for the top-tier, high-performance die in the consumer lineup

Die — In semiconductor manufacturing and computer architecture, a die is a small, rectangular block of semiconducting material (usually silicon) on which a specific functional circuit - such as processor or memory - is fabricated.

Key characteristics of GA102

Architecture — NVIDIA Ampere (8nm process).

Transistor — Approximately 28.3 billion.

Die size :- 628 mm^2

Core components — features Graphics processing clusters (GPCs), streaming Multiprocessors (SMs), RT cores (Ray Tracing), and Tensor Cores (AI).

Transistor — Transistors are the fundamental, microscopic semiconductor switches in computer architecture, acting as the building blocks for logic gates, processors (CPUs / GPUs) and memory.

By switching electrical signals on (1) or off (0) billions of times per second, they enable binary computation, data storage, and processing, replacing bulky vaccum tubes to allow for fast, energy-efficient, and dense integrated circuits.

Fundamental Role: They act as binary switches, allowing electrical current to flow (on) or blocking it (off), representing 1s and 0s.

Building Logic Gates: Transistors are combined to form logic gates (AND, OR, NOT), which perform Boolean operations and arithmetic.

Core components: Billions of transistors are packed into modern chips (CPUs, GPUs, RAM) using Silicon technology.

Moore's law & Scaling: The ability to shrink transistors (e.g., 3nm processes) allows for higher transistor counts, such as the 184 billion found Apple's M3 Ultra, directly increasing computational power.

Types: MOSFETs (Metal-Oxide-Semiconductor field-Effect Transistors) are the primary type used in modern digital logic for their efficiency as switches.

GPC – Graphics Processing Cluster (GPC) is a proprietary term and architectural unit specific to NVIDIA.

In NVIDIA GPU computer architecture, a Graphics Processing Cluster (GPC) is a top-level hardware unit responsible for distributing graphics and compute workloads across its internal resources. It acts as major, independent processing block that combines multiple streaming Multiprocessors (SMs) and specialized rendering hardware, crucial for parallel processing in gaming, rendering and AI workloads.

GPC

A GPC is a "self-contained" block within the GPU die. A single GPC typically contains:

Raster Engines: Responsible for transforming 3D geometry (triangles) into 2D pixels (rasterization).

Texture Processing Clusters (TPCs): A subset of the GPC, which contains:

Streaming Multiprocessors (SMs): where the actual CUDA core computations (FP32, INT32) take place.

PolyMorph Engines: Handle vertex fetching, tesselation, and viewport transformation.

A tesselation is a pattern of one or more shapes that covers one surface without gaps or overlaps, forming a repeating, tile-like structure.

Tensor Cores & Ray Tracing Cores: In modern architectures, these reside within the SMs inside the GPC.

ROPs (Raster Operator Partitions): In newer architectures like Ampere (e.g., GA102), ROPs are integrated into the GPC, which helps eliminate bottlenecks.

Role in GPU Hierarchy: The GPC sits near the top of the GPU compute hierarchy:

1. GPU (chip)
2. Graphics Processing clusters (GPCs)
3. Texture Processing clusters (TPCs)
4. Streaming Multiprocessors (SMs)
5. CUDA cores.

A high-end GPU (e.g., RTX 3090, H100) contains multiple GPCs to scale performance.

Key characteristics:

Parallelism & Scalability: More GPCs generally mean higher parallel compute and graphics capability.

Indipendency: Each GPC operates independently, allowing the GPU to manage different parts of a workload simultaneously.

Evolution: In architectures like NVIDIA Turing and Ampere, the GPC structure was refined to increase throughput, particularly by integrating ROP units directly into the GPC, allowing for higher, faster rendering and computations.

In short, GPCs are the fundamental, scalable "workhouse" blocks that enable modern GPUs to perform massively parallel tasks, from rendering complex 3D scenes to training AI models.

Streaming Multiprocessors (SMs) -

A Streaming Multiprocessor (SM) is the fundamental, core building block of modern NVIDIA GPUs responsible for executing instructions in parallel.

It functions as a highly threaded processor capable of managing hundreds of threads simultaneously, designed to maximize throughput for graphics rendering and General-Purpose GPU (GPGPU) compute workloads.

A single GPU typically contains multiple SMs, such as the 170 SMs found in the RTX 5090.

Core components of an SM

Modern SMs (e.g., Ampere / Hopper architectures) are composed of several specialized components:

CUDA Cores (Stream Processors): These are the basic Arithmatic Logic Units (ALUs) that perform integer (INT32) and floating-point (FP32) calculations, with roughly 128 cores per SM in newer architectures.

Tensor Cores: Specialized units for AI and deep learning, optimized for matrix multiplication and accumulation (4×4 matrix ops) with high throughput.

RT Cores: Hardware units dedicated to ray-triangle intersection testing and bounding volume hierarchy (BVH) traversal for accelerating ray tracing.

Warp Schedulars & Dispatch Units: Control thread execution by picking ready warps (groups of 32 threads) and issuing instructions to execution units.

Register files: Extremely fast, large storage area holding thread-specific data, enabling fast context switching.

Shared Memory / L1 Cache: Programmable, low-latency memory used for data sharing between threads, typically configurable to manage bandwidth efficiently.

Special function Units (SFUs): Handle complex calculations like trigonometric functions and square roots.

Execution Model : SIMT and Warps :

SMs operate using a Single Instruction, Multiple Thread (SIMT) model.

Warps: When a kernel (a parallel program) is launched, the GPU breaks it down into thread blocks, which are then split into 32-thread units called warps.

Lock-step Execution: Threads within a warp execute the same instruction simultaneously, though individual threads can be masked off (disable) during branch divergence.

Latency Hiding: SMs hide memory latency by instantly switching between active warps. When one warp stalls for memory access, the scheduler immidiately picks another ready warp to execute, ensuring the CUDA cores stay busy.

Key Performance concepts:

Occupancy: The ratio of active warps to the maximum allowed warps per SM. High occupancy is crucial to covering memory latency but must be balanced against register usage.

Scalability: GPU performance scales with the number of SMs. A higher number of SMs means more parallel thread can be processed simultaneously.

Register Pressure: Because each SM has a fixed amount of register space, using too many registers per thread limits the number of active threads (occupancy), potentially reducing performance.

Evolution in Architecture

Pre-Unified (G80): Early GPUs had separate processors for vertex and fragment shaders.

Unified (Tesla, 2006): Introduced the SM, allowing a single unit to run different shader types (vertex, geometry, fragment) and general-purpose

compute, revolutionizing GPU flexibility.

Modern (Ampere / Hopper): Enhanced with Tensor cores for AI, RT cores for ray tracing, and specialized asynchronous data transfer instructions to maximize throughput.

Warp —

Warps are typically unit of execution on a GPU

"Warp is a bundle of threads to complete a particular GPU task where thread is just a piece of code execution."

Self ↗

In computer architecture, a warp is a group of threads that execute the same instruction at the same time on parallel hardware — mainly used is SIMT (Single Instruction, Multiple Threads) design.

Instead of GPU managing thousands of threads individually, it manages them in warps to stay efficient.

How it works (conceptually)

- Many threads are launched.
- Hardware bundles them into fixed-size groups → warps
- Each warp executes one instruction per cycle across all its threads.
- If threads inside a warp take different branches → warp divergence occurs → which leads to slower execution.

Typical characteristics

- Fixed number of threads per warp (often 32 in NVIDIA architectures)
- Scheduled as single unit
- Shares instruction flow
- Improves parallel throughput and simplifies scheduling.

Why architectures use warps

- Reduces scheduling overhead
- keeps execution unit busy
- Makes massive parallelism manageable
- Hides memory latency by switching between warps.

Important

Threads = individual workers

warp = a synchronized squad

GPU core = supervisor giving one instruction to the whole squad.

Key Aspects of Warps in GPU Architecture

Structure and Size: In NVIDIA GPUs, a warp consists of 32 threads. Threads within a block are divided into these 32-wide bundles.

Execution (SIMT): All threads in a warp execute the same instruction simultaneously, though they may operate on different data.

Warp Scheduling: Multiple warps are managed by an SM's scheduler, which determines which executes next to keep the execution units busy and hide memory latency.

Divergence: If threads within a warp take different execution paths (e.g., an if-else statement), the warp suffers from "warp divergence", where the paths are executed sequentially rather than in parallel, reducing efficiency.

Performance optimization: To ensure high performance, developers are advised to align thread blocksizes to multiples of warp size (32) and minimize branch divergence within code.

Warp vs other Terminologies

Wavefront: Used by AMD as an equivalent to NVIDIA's "warp", often with a size of 64 threads.

Warp Computer (Historical): A specialized systolic array processor developed by Carnegie Mellon University, which is different from the modern GPU "warp".

Specialized Systolic Array: A specialized systolic array processor is a highly efficient, parallel computing architecture designed as an application-specific hardware implementation for algorithms with regular data dependencies, such as matrix operations and digital signal processing.

Ray Tracing:

A ray tracing core (often called an RT core) is a specialized hardware unit inside modern GPUs designed specifically to accelerate ray tracing calculations — the math used to simulate realistic light, shadows, and reflections.

Ray tracing is a rendering technique where the computer simulates how light travels in a virtual 3D scene by mathematically tracing paths of light rays to calculate what each pixel should look like.

What really happens under the hood

Instead of drawing triangles directly (like rasterization), the render:

1. Defines a virtual viewpoint (observer position in the scene).
2. Generates one or more mathematical rays per pixel.
3. Tests where those rays intersect scene geometry.
4. Calculates lighting based on:
   - material properties
   - light source
   - reflections
   - shadows
   - indirect light bouncing
5. Produces final pixel colors.

Hardware like RT cores on GPUs from NVIDIA accelerates the intersection math.

Architecturally

Ray Tracing is basically:

- massive parallel geometric intersection testing.
- recursive or iterative light transport simulation.
- probabilistic sampling (especially in path tracing).

Conceptual flow

pixel → generate ray → intersect geometry → evaluate lightning → write colour.

What Ray Tracing actually does

Instead of traditional raster graphics, ray tracing:

1. Shoots virtual ray from the camera

This camera in ray tracing:

- a position + direction + fied of view
- inside a virtual 3D world
- created by a game engine or graphics program
  - → a game engine
  - → a 3D renderer
  - → a simulation program
  - → GPU memory during rendering

2. Checks what objects those rays hit
3. Calculates lighting, reflections, shadows, refraction.

This requires massive geometry intersection tests — extremely expensive computationally.
RT cores exists to make those calculations fast.

What a RT core does in hardware?

Main acceleration tasks:
1. Bounding Volume Hierarchy traversal (BVH)
   - Scene objects organized into tree structures. (Scene Graph)
   - RT cores quickly walk through the tree.
   - Skip objects rays cannot hit
2. Ray-triangle intersection tests
   - Determines if a ray hits a triangle surface
   - Done millions or billions of times per frame.
3. Ray-box intersection
   - Tests if ray intersects bounding volumes
   - Helps early rejection of geometry.

So instead of general SM/CUDA cores doing everything, RT cores handle these specific geometric operations.

Where RT cores sit in architecture

Typical modern GPU structure:

- SM (Streaming Multiprocessor) → general compute
- Tensor cores → AI / matrix math
- RT Cores → ray tracing acceleration

The SM launches ray tracing workloads, and RT cores handle the heavy intersection math.

Why RT cores matter

Without RT cores:

- Ray Tracing is painfully slow
- Requires huge computing budgets

With RT cores:

- Real-time reflections
- Global illumination
- Accurate shadows
- Realistic lighting in games & simulations.

Mental Model

SM = general-purpose worker
Tensor core = AI math specialist
RT core = geometry intersection expert.

Important nuance

"Ray tracing core" isn't a universal CPU architecture concept - it's a GPU-specific hardware accelerator found in modern architectures.